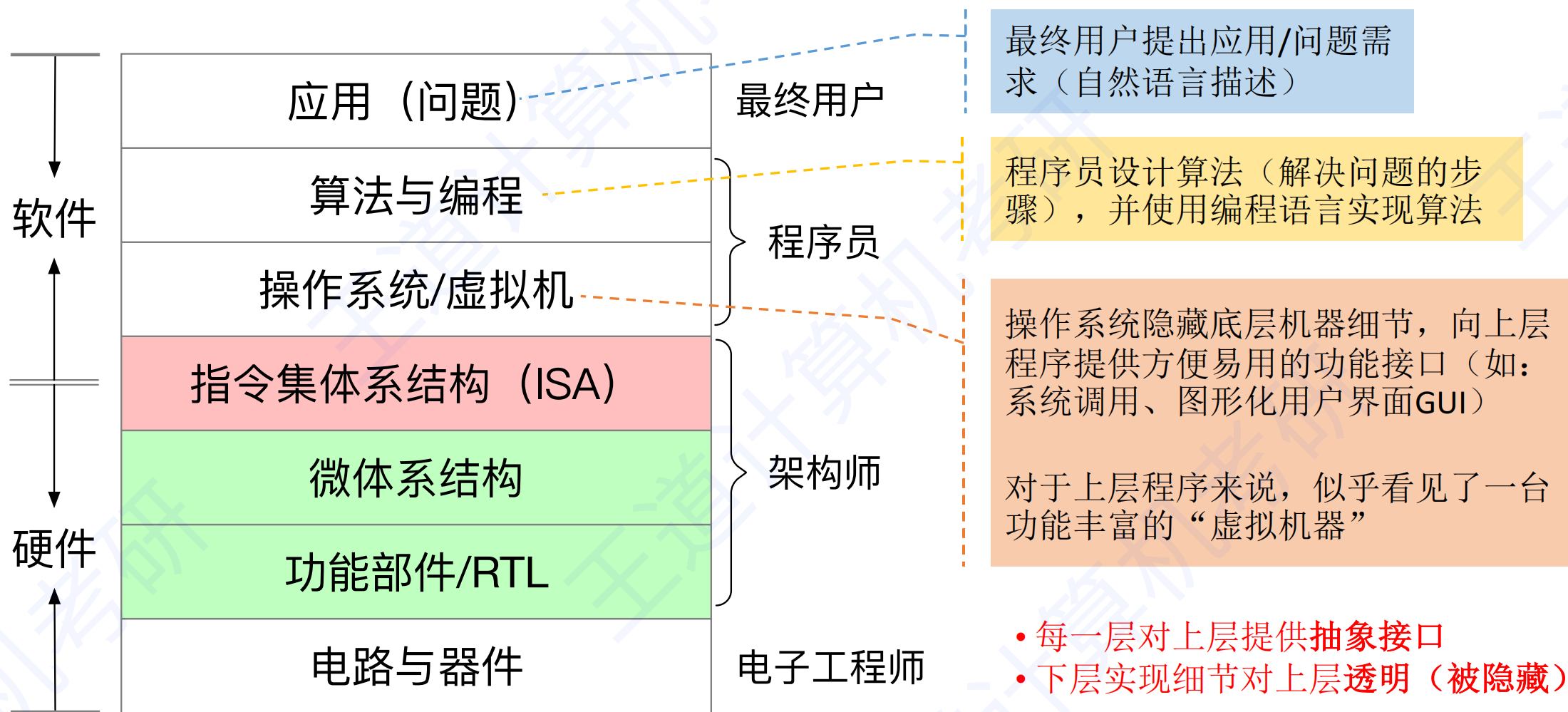


本节内容

计算机系统 层次结构

计算机系统的层次结构



高级、低级 编程语言

低级语言 (low-level language) 和运行程序的计算机的底层结构密切相关，也称为**机器级语言** (machine-level language)

高级语言 (high-level language) 和计算机底层结构无关，人类可读性强

机器语言：二进制代码
0/1 指令序列（可读性差）

汇编语言：助记符

机器语言的符号化表示，
汇编语言和机器语言语句
一一对应（可读性提升）

高级语言：C/C++、Java、Python

CPU可以
直接执行

机器语言程序

```
0000010000000101
0001000000000110
.....
```

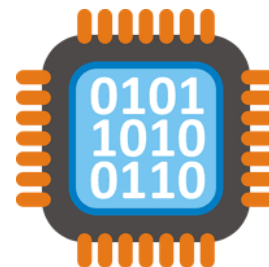
汇编程序
(汇编器)

```
LOAD 5
MUL 6
.....
```

编译程序
(编译器)

```
y = a*b+c
.....
```

源程序



三类翻译程序（高级→低级语言）

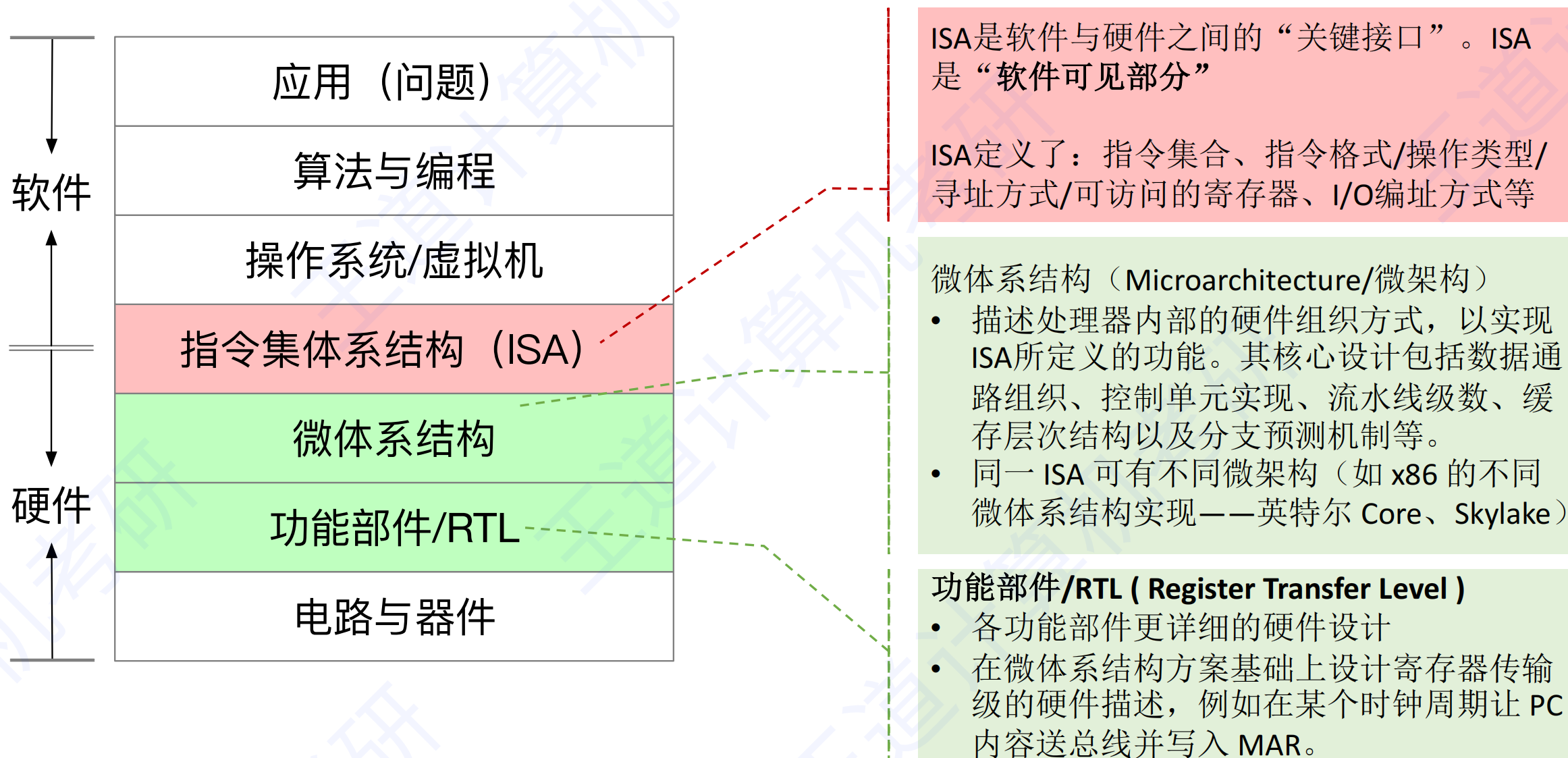
- **汇编器**：汇编 → 机器语言
- **解释器**：逐条翻译并立即执行
- **编译器**：高级 → 汇编/机器语言

编译程序
(编译器)

解释程序
(解释器)

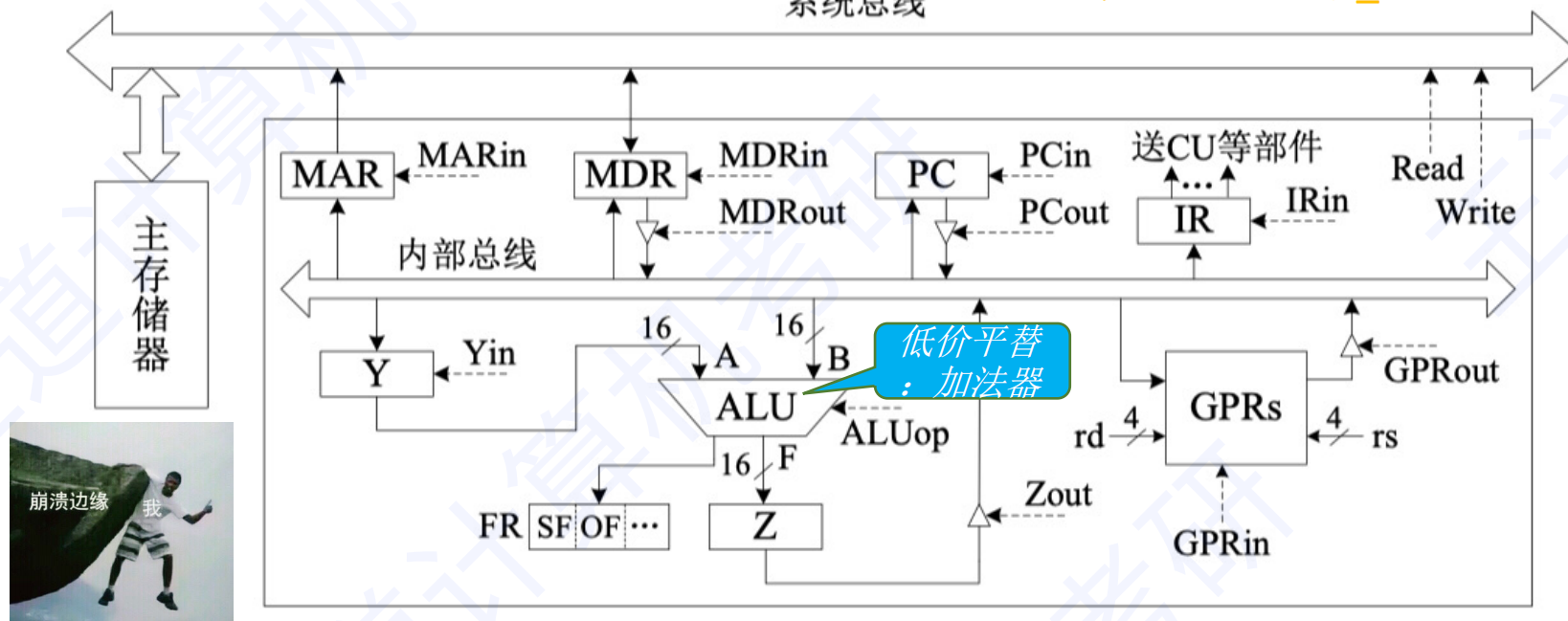
```
0000010000000101
0001000000000110
...
```

计算机系统的层次结构



ISA→微体系结构→功能部件/RTL

↓注：2022年_43题图



ISA层设计范畴——硬件应支持哪些指令，各指令功能是什么？例如：

- ADD R1, R2, R3 #加法指令， $R2 + R3 \rightarrow R1$
- SUB R1, R2, R3 #减法指令， $R2 - R3 \rightarrow R1$
- MUL R1, R2, R3 #乘法指令， $R2 \times R3 \rightarrow R1$
- DIV R1, R2, R3 #除法指令， $R2 \div R3 \rightarrow R1$
- LOAD R1, addr #取数指令， $[addr] \rightarrow R1$
- STORE R1, addr #存数指令， $R1 \rightarrow [addr]$

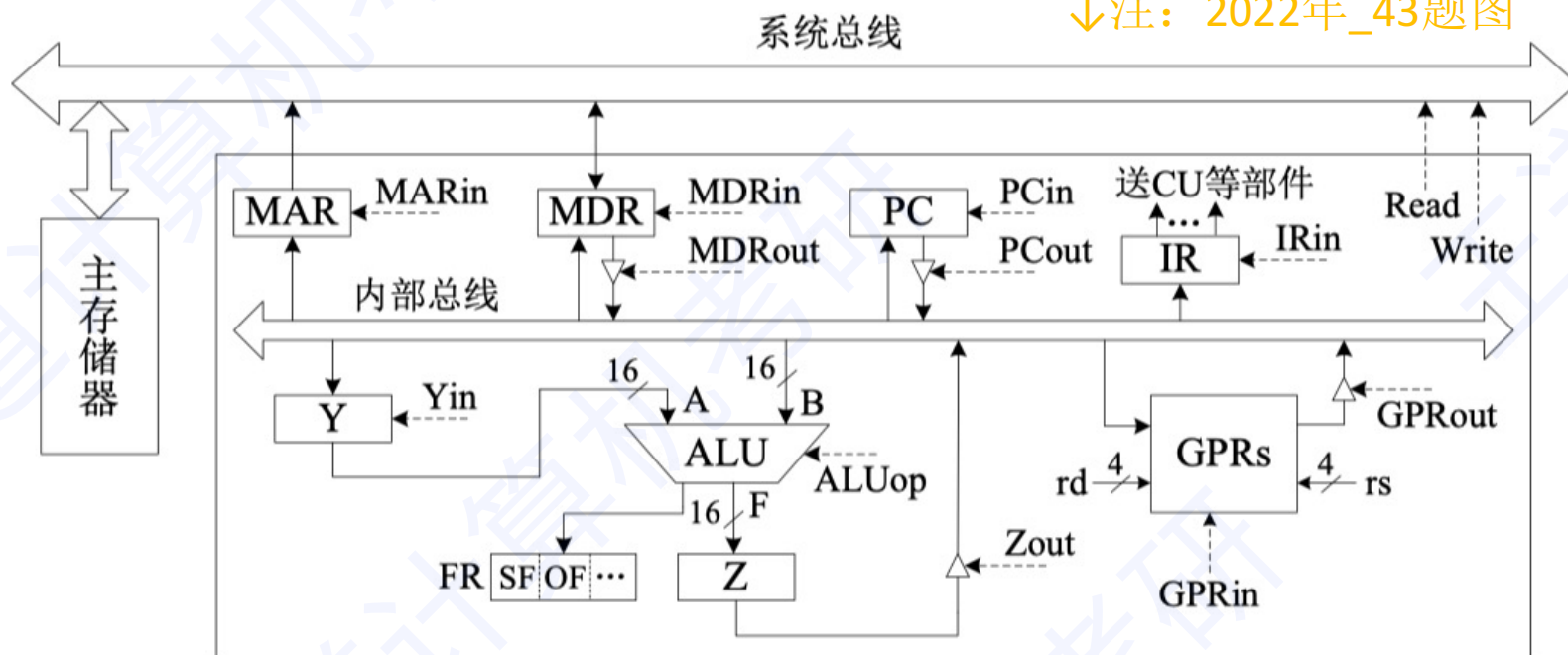
微体系结构层设计范畴——应该使用哪些功能部件、如何组织这些功能部件。例如：

- 使用一个支持加/减/乘/除运算的ALU
- CPU内部采用单总线结构、配合暂存寄存器Y、Z
- 使用一个寄存器组GPRs
- 使用系统总线连接主存与CPU
- 各部件之间的逻辑连接、数据通路设计

若ISA层只要求支持加法运算，则ALU可以替换为串行进位加法器或并行进位加法器

ISA→微体系结构→功能部件/RTL

↓注：2022年_43题图



微体系结构层设计范畴——应该使用哪些功能部件、如何组织这些功能部件。例如：

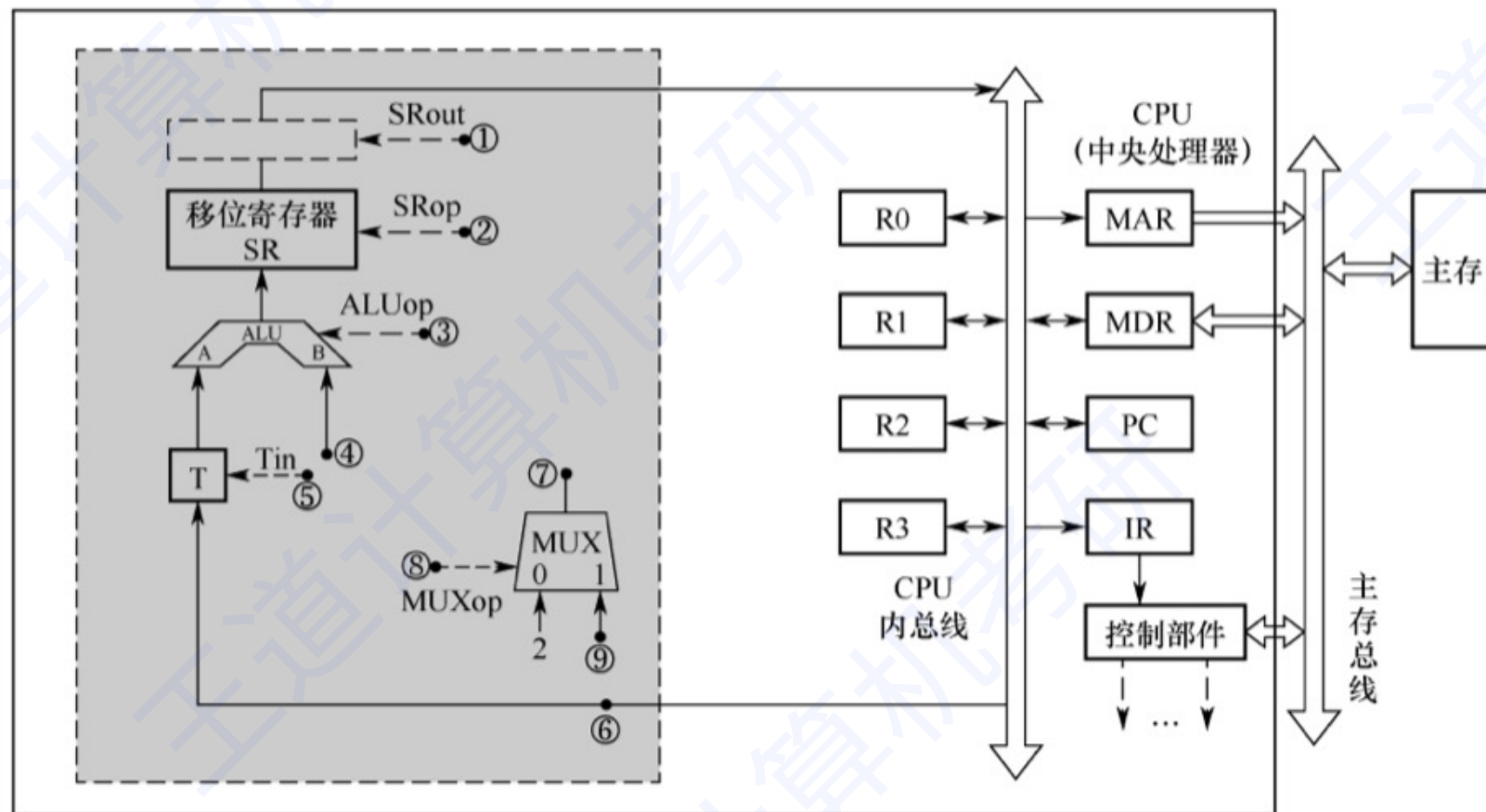
- 使用一个支持加/减/乘/除 运算的ALU
- CPU内部采用单总线结构、配合暂存寄存器Y、Z
- 使用一个寄存器组 GPRs
- 使用系统总线连接主存与CPU
- 各部件连接逻辑、数据流动设计



- 功能部件/RTL 设计范畴——各功能部件更详细的硬件设计；RTL = **Register Transfer Level**（寄存器传输级别）核心关注的是在每个时钟周期中，数据从哪个寄存器出来，经过什么部件，最后写入哪个寄存器。例如：
- 每个功能部件有几个引脚、各个引脚分别有什么作用
- GPRs 内部要如何连接译码器与多路选择器（本题3小问）
- RTL：比如当 CU 发出 PCout 和 MARin 有效时，数据在下一个时钟滴答声中，从 PC 流出，穿过内部总线，流入 MAR。

真题示例：2015年真题_43、44

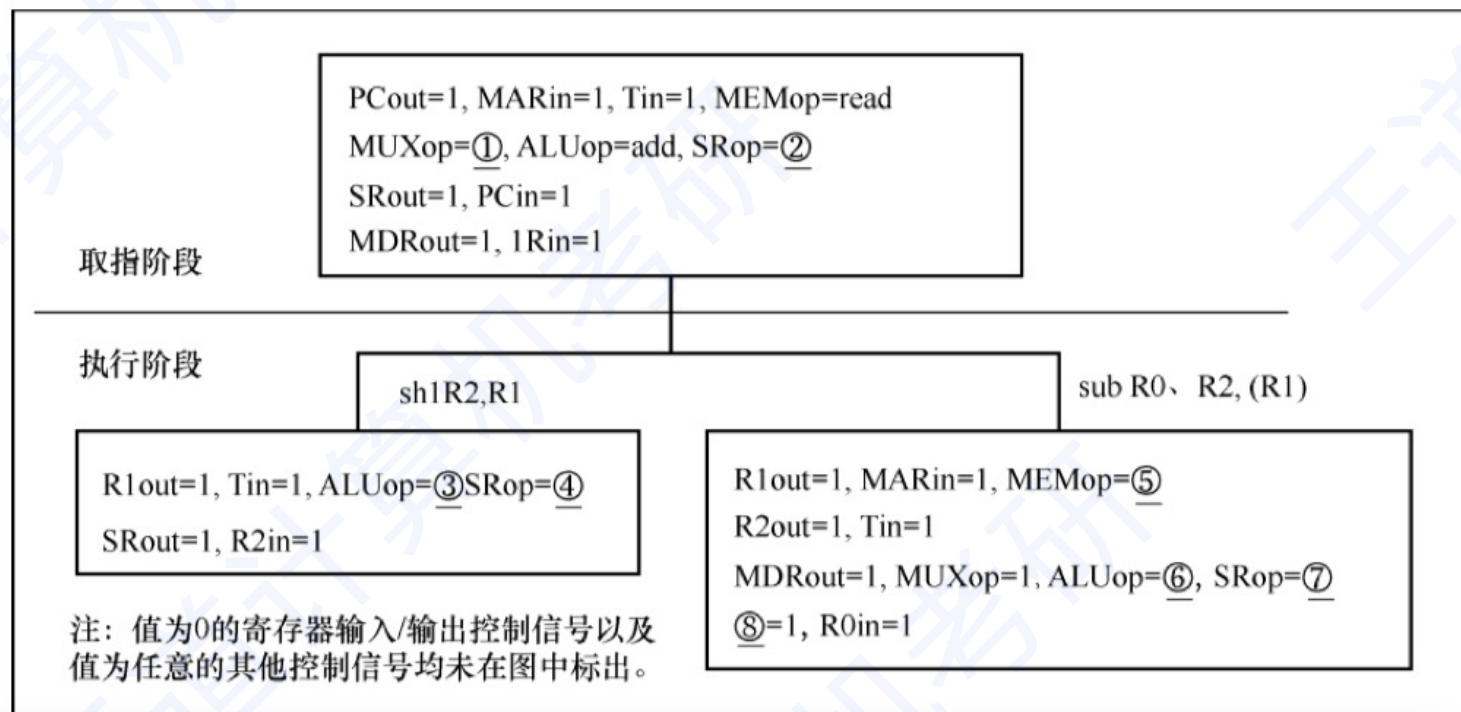
↓注：2015年_43题图



该题题干给出ISA层的一些设计规定（如：指令字长等），并要求考生设计部件之间的逻辑连接、选择虚线框内缺失的功能部件，设计功能部件所需的控制信号等。是典型的“**微体系结构**”设计。

真题示例：2015年真题_43、44

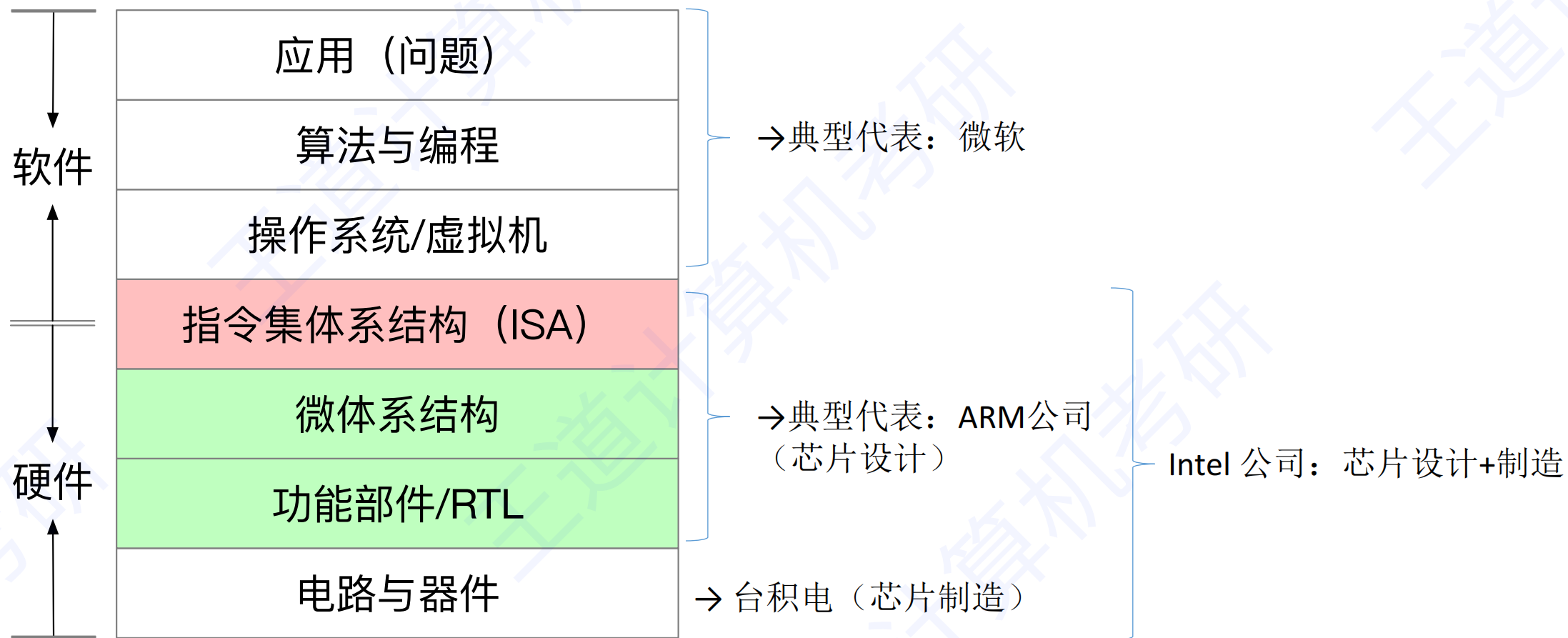
↓注：2015年_44题图a



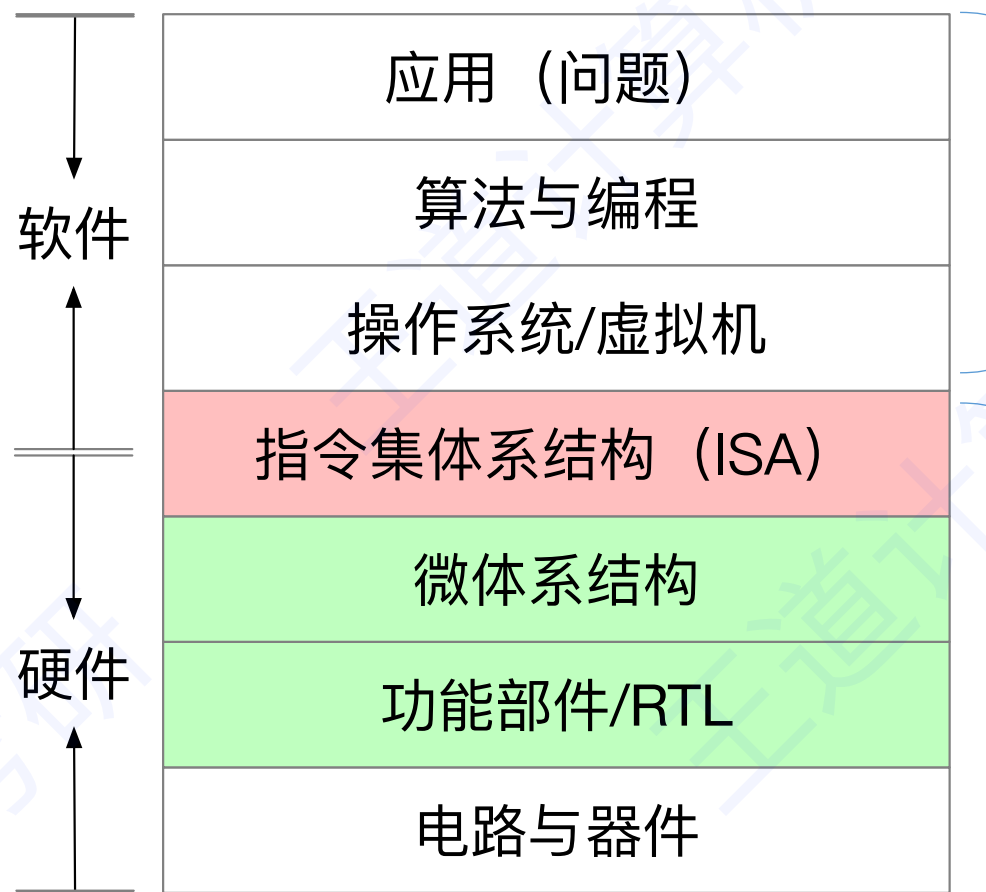
3) 假设寄存器 X 的输入和输出控制信号分别为 Xin 和 Xout, 其值为 1 表示有效, 为 0 表示无效 (如 PCout = 1 表示 PC 内容送总线); 存储器控制信号为 MEMop, 用于控制存储器的读 (read) 和写 (write) 操作。写出题图 a 中标号①~⑧处的控制信号或控制信号的取值。

该题 3) 小问要求考生基于43题的**微体系结构**, 设计左移指令、减法指令 在每个时钟周期的寄存器传送和控制信号, 是典型的 **RTL** 层面的描述。

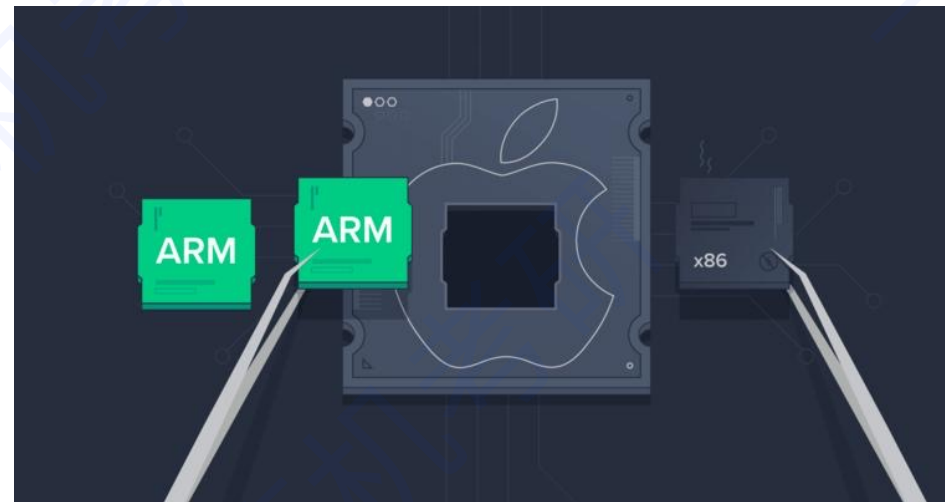
计算机系统的层次结构（典型代表公司）



2020年苹果PC业务换芯事件：x86 → ARM



→苹果公司PC业务（2020年以前，苹果电脑使用英特尔芯片、x86指令集）



Intel 公司：芯片设计+制造

2020年苹果PC业务换芯事件：x86 → ARM



Rosetta 2 自动转译 (最关键的过渡手段)——这是苹果处理旧款 Intel x86_64 应用的主要手段。Rosetta 2 是一个动态二进制翻译器。它在用户首次运行 Intel 应用时, 自动将 x86_64 指令集转换为 ARM64 指令集。(用户无感知, 丝滑切换至全新 ISA)

2020年苹果PC业务换芯事件：x86 → ARM



2026年推出的 MacBook Neo 使用 A 系列芯片，与苹果公司将 Mac 从 Intel x86 架构迁移至 ARM 架构的战略转变有着直接且深层的关联



注：事实上，苹果手机业务的 A 系列芯片很早就在使用 ARM ISA 了，因此 PC 业务芯片转向 ARM，有助于软件生态“大统一”。